

Combating Concept Drift with Explanatory Detection and Adaptation for Android Malware Classification

Yiling He

The State Key Laboratory of
Blockchain and Data Security,
Zhejiang University
Hangzhou, China
heyilinge0@gmail.com

Junchi Lei

The State Key Laboratory of
Blockchain and Data Security,
Zhejiang University
Hangzhou, China
junchilei@zju.edu.cn

Zhan Qin

The State Key Laboratory of
Blockchain and Data Security,
Zhejiang University
Hangzhou, China
qinzhao@zju.edu.cn

Kui Ren

The State Key Laboratory of
Blockchain and Data Security,
Zhejiang University
Hangzhou, China
kuiren@zju.edu.cn

Chun Chen

The State Key Laboratory of
Blockchain and Data Security,
Zhejiang University
Hangzhou, China
chenc@zju.edu.cn

Abstract

Machine learning-based Android malware classifiers struggle with concept drift: the rapid evolution of malware, especially with new families, can depress classification accuracy to near-random levels. Previous research has largely centered on detecting drift samples, with expert-led label revisions on these samples to guide model retraining. However, these methods often lack a comprehensive understanding of malware concepts and provide limited guidance for effective drift adaptation, leading to high human labeling costs.

To combat concept drift, we propose DREAM, a novel system that establishes an explanatory drift detection and adaptation process. Our core idea is to integrate classifier and expert knowledge within a unified model. To achieve this, we embed malware behavioral concepts within the latent space of a contrastive autoencoder, while constraining sample reconstruction based on classifier predictions. This approach enhances retraining in two key ways: 1) capturing the target classifier's characteristics to select more effective samples in detection and 2) enabling concept revisions that extend the classifier's semantics to provide explainable guidance for adaptation. Additionally, DREAM eliminates reliance on training data during real-time drift detection and provides a behavior-based explainer to support concept revision. Our evaluation shows that DREAM effectively improves the drift detection accuracy and reduces the expert analysis effort in adaptation across different malware datasets and classifiers. Notably, when updating a widely-used DREBIN classifier, DREAM achieves the same accuracy with 76.6% fewer newly labeled samples compared to the best existing methods.

CCS Concepts

• Security and privacy → Software and application security.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CCS '25, Taipei

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1525-9/2025/10
<https://doi.org/10.1145/3719027.3744792>

Keywords

Concept Drift; Malware Analysis; Explanatory Interactive Learning

ACM Reference Format:

Yiling He, Junchi Lei, Zhan Qin, Kui Ren, and Chun Chen. 2025. Combating Concept Drift with Explanatory Detection and Adaptation for Android Malware Classification. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25)*, October 13–17, 2025, Taipei. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3719027.3744792>

1 Introduction

Android malware classification is continually challenged by concept drift [55]. As cyber attackers constantly devise evasion techniques and varied intents [5], the evolving nature of malware behaviors can rapidly alter the patterns on which classifiers rely. Consequently, static machine learning models trained on historical data face a significant drop in performance and become incapable of handling unseen families [29]. To combat malware concept drift, current state-of-the-art solutions leverage active learning [32], comprising two primary stages. In the *drift detection* stage, new test samples that exhibit signs of drift are periodically selected. Most existing research has focused on improving this stage using techniques like statistical analysis [7, 37] or contrastive learning [15, 78]. The subsequent *drift adaptation* stage follows a standard approach: the identified drifting samples are labeled by malware analysts and then added to the training set for classifier retraining [80].

Existing methods for drift detection fall short in two main aspects. Firstly, they often falsely neglect the patterns on which the targeted classifier depends. For example, CADE [78] leverages an independent autoencoder to learn a distance function, identifying drifts by the distances from training data. Such misalignment can lead to ineffectiveness especially when dealing with complex classifier feature spaces, as validated by our experiments in Section 5.2. Secondly, existing methods often rely on constant access to training data during the testing phase, such as querying reference uncertainties [7, 37] or searching for nearest neighbors [15], which would bring storage and security concerns in local deployment [59].

	Detection		Adaptation		Application		
	MS	DA	Act.	Exp.	Inter	Intra	Agn.
Trans. [7]	●	○	○	○	●	●	●
CADE [78]	○	●	●	○	●	●	○
HCC [15]	●	○	●	○	○	●	○
DREAM	●	●	●	●	●	●	●

^a ●=true, ○=false, ◐=partially true.

^b Detection: distinguished by model sensitivity (MS) and data autonomy (DA). Adaptation: can be proactive and explanatory. Different support in the detection and proactive adaptation phases: inter-class drifts, intra-class drifts, and model-agnostic applications.

Table 1: Comparison to existing work in terms of the detection method, adaptation method, and the applicable scope.

In the drift adaptation stage, the common label-centric retraining strategy poses significant challenges to human efforts. A clear discrepancy exists between the extensive analysis required for assigning labels and the limited information actually utilized by the model. While analysts use static/dynamic program analysis to extract rich reasoning insights [2, 46], these insights are largely disregarded as the model only considers the revised labels. This mismatch limits the effectiveness of model updates, with mislabeling further complicating improvements [56, 73]. Consequently, analysts often need to label a large number of samples to ensure accurate updates.

To address these challenges, our high-level idea is to design a system that aligns closely with the classifier knowledge while supporting human revision of malware behavioral concepts. This alignment facilitates the two integral processes: 1) in drift detection, deviations are identified by assessing the reliability of concepts under the supervision of the classifier; 2) in drift adaptation, experts are provided with interfaces to directly revise the deviated concepts, enabling the adjustments to inversely update the classifier.

We develop the system, called DREAM, distinguished by model-sensitive drift detection and explanatory adaption for Android malware classification. Specifically, we train an autoencoder with model sensitive concept learning. To embed malware concepts, we combine supervised concept learning [35] with unsupervised contrastive learning [14]: elements in the latent space are representative of behavioral concepts that can be labeled, such as remote control and stealthy download. For drift detection, DREAM integrates classifier knowledge and achieves data-independent testing through a unique concept reliability measure, grounded in the intuition that reconstructed samples from reliable concepts should yield consistent model predictions with the original samples.

For adaptation, we envision that human intelligence can not only be utilized for feedback on prediction labels but also for the explanations [67]. Rather than creating an external explanation module for the classifier, we utilize concepts embedded within our detector. This integration offers dual benefits: high-level explanation abstraction for human understanding [30] and immediate revision impact on evolving concepts. Furthermore, DREAM effectively harnesses human intelligence with a joint update scheme. The classifier's retraining process is assisted by the detector, allowing for the utilization of both malware labels and behavioral explanations.

We evaluate DREAM with 2 distinct Android malware datasets [71, 78] and 3 state-of-the-art classifiers [3, 47, 50]. Results demonstrate that DREAM outperforms 3 existing drift detection methods [7, 15, 78] in malware classification, achieving an average

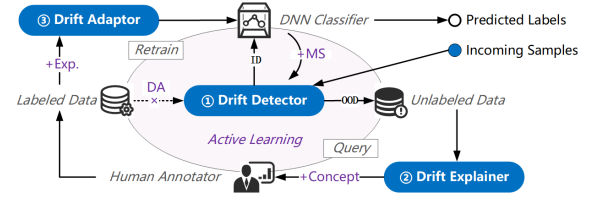


Figure 1: Active learning framework for concept drift mitigation: standard components and operations in common active learning (grey, italic), specialized components for concept drift (blue boxes), and new features of our method compared to existing works (purple).

improvement of 11.5%, 12.0%, and 13.6% in terms of the AUC¹ metric. For drift adaptation, DREAM notably enhances the label-centric retraining approach. It achieves improved F1-scores of 168.4%, 73.5%, 22.2%, 19.7%, and 6.0% over the benchmark under the same data constraints when the human analysis budget is set at 10, 20, 30, 40, and 100, respectively. A standout observation is the increased advantage at lower human labeling costs. For instance, analysts can analyze over 80% fewer samples while still achieving a test accuracy of 0.9 for the updated classifier.

Contributions. This paper has three main contributions.

- We revisit existing drift detection methods in malware classification, identifying necessity for model sensitivity and data autonomy. We design a novel drift detector to meet the two needs, enriched with the integration of malware behavioral concepts.
- We initiate an explanatory concept adaptation process, enabling expert intelligence to revise malware labels and concept explanations. Through a joint update of both the classifier and our drift detector, we harness the synergy between human expertise and automated detection.
- We implement DREAM and evaluate it across different datasets and classifiers. Experimental results show that DREAM significantly outperforms existing drift detectors and the conventional retraining-based adaptation approach.

2 Background

In this section, we delve into the background of malware concept drift. We begin by discussing the two primary types of drift in learning-based malware classifiers, particularly in the context of Android. Next, we introduce existing work for mitigating the drift, highlighting key studies for dynamic adaptation in this domain.

2.1 Android Malware Concept Drift

The widespread adoption of Android, which holds a dominant mobile market share of 71.31% as of April 2024 [33], has made it an attractive target for cybercriminals [16]. With over 3 million new malware samples detected every month [4], the platform faces a rapidly evolving threat landscape. This dynamic environment, driven by attackers seeking to evade [36, 70], poses significant challenges for both deep learning-based malware detection and classification systems [21, 24, 42, 57]. These systems are particularly vulnerable to *concept drift*, a phenomenon where the statistical

¹The AUC specifically refers to the Area Under the ROC Curve [20] in this paper.

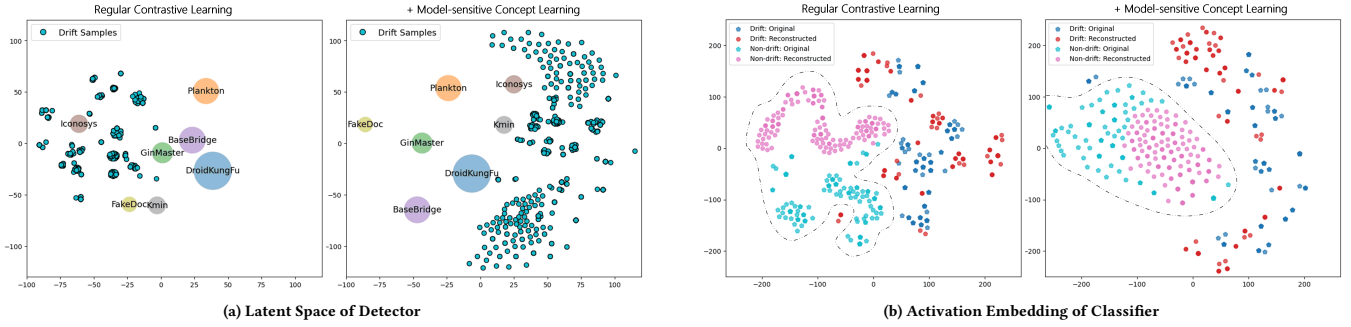


Figure 2: Model sensitive concept learning: how concepts are learned by the detector and used by the classifier.

properties of data change over time, leading to a degradation in model performance [26, 55].

Concept drift in malware manifests in two primary forms, with the ever-evolving nature of Android malware providing prominent examples: 1) *Intra-class drift* refers to changes within existing malware families, such as the emergence of new variants. For example, the information-stealing Android malware Xavier produced 5 different versions within eight months of its discovery [74]. This type of drift primarily impacts the binary detection tasks, with studies showing that models for Android malware can degrade to near-random levels after two years without updates [29, 52]; 2) *Inter-class drift*, which is the focus of this paper, involves the emergence of entirely new malware families. The rapid emergence is exemplified by the identification of 10 new families of Android banking malware alone in 2023, each targeting critical financial applications globally with different capabilities [68]. This type of drift is pressing, as it requires timely updates to the multi-class classification models to ensure a more granular response to malware [23].

2.2 Mitigating Malware Concept Drift

Traditional approaches to mitigating malware concept drift often lack support for online scenarios that require continuous adaptation. These methods typically involve redesigning feature spaces, followed by retraining from scratch using the new features [8, 11]. The key challenge in this process is identifying a feature set that remains relevant as the malware evolves. A recent work proposes drift forensics [18], adopting multiple retraining and feature attribution calculations across different dataset divisions. While their in-depth analysis provides valuable insights for feature selection in a binary feature space [3], their method relies on ground truth labels and become infeasible for complex classifiers. Designing robust feature spaces remains an open research question. To address online scenarios, recent research has adopted a two-stage paradigm: determining when to take action (*drift detection*) and what action to take (*drift adaptation*).

Drift Detection. The detection stage involves identifying drift by selecting ambiguous samples. Much of the existing research focuses on this stage, drawing inspiration from Out-of-Distribution (OOD) detection methods in the machine learning domain [77]. For example, *uncertainty estimation* gauges a model’s prediction confidence, with a naive approach for DNN being the probability.

However, probability alone can be misleading, as overfitted models may assign high softmax values to unfamiliar data, falsely indicating high confidence [31, 54]. As a result, many researchers have shifted towards *nonconformity scoring*, which evaluates how unusual new data is compared to a calibration set [7, 37]. Notably, *contrastive learning* has shown promise in modeling an effective distance function for this purpose [15, 78].

Drift Adaptation. Once drift is detected, the goal is to properly handle the identified samples. A conservative approach involves withholding ambiguous samples for expert analysis to minimize misclassification risks [7, 37]. While this strategy provides a safeguard against immediate threats, it lacks long-term resilience. Alternatively, *active learning* (AL) inspired strategies introduce a labeling budget to prioritize a subset of drifting samples for humans to label. These labeled samples are then integrated into the retraining process, enabling the model to adapt dynamically to new data distributions [15]. To streamline human labeling in active learning, there’s growing interest in *drift explanation* [28, 78], but how these explanations can help update the classifiers remains underexplored.

Summary of Key Studies. Advancements in drift detection and adaptation techniques are essential for combating malware drifts dynamically. As summarized in Table 1, three notable existing studies have contributed significantly to the field. Among these, Transcendent focuses on drift detection by refining the calibration process and employs a conservative adaptation approach, whereas CADE and HCC use contrastive learning for drift detection and active learning for adaptation. Notably, HCC specializes in intra-class drift by incorporating a hierarchical structure into the contrastive loss, and its drift detector has an inherent design that is incompatible with off-the-shelf classifiers. This paper targets a similar scope of applicability as CADE but also addresses inter-class adaptation, a challenging aspect not considered in these existing works.

3 Motivation and Research Scope

In this section, we identify the limitations of current drift detection methods and active learning-based drift adaptation strategies, followed by a discussion of our research scope.

3.1 Limitations on Current Detection Methods

Current inter-class drift detectors often lack *model sensitivity* and *data autonomy*, resulting in ineffective drift detection.

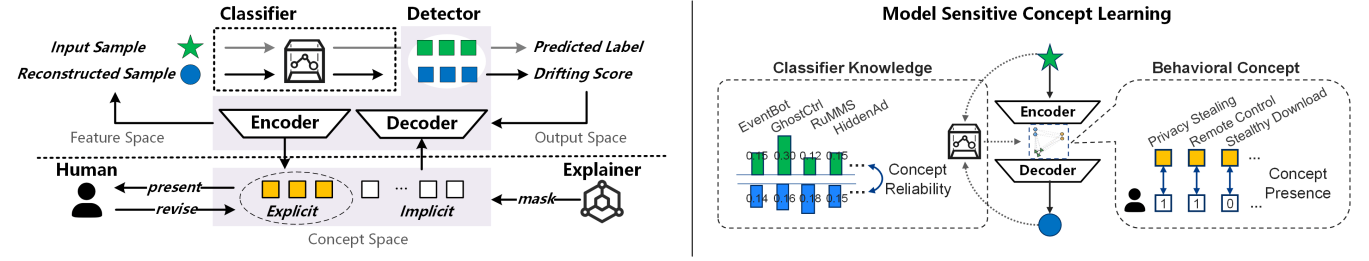


Figure 3: Overview (left) and design insights (right) of our DREAM system. To enhance contrastive autoencoder-based drift detector, we employ model sensitive concept learning: DREAM enriches the latent space with concepts of malicious behaviors, and it learns those concepts along with classifier knowledge.

(L1) Firstly, these detectors are typically trained independently of the classifier, which should have been the subject of drift. For instance, the contrastive learning-based methods [15, 78] use an unsupervised approach to learn a distance metric with training data, without fully considering the specific characteristics of the classifier. This unguided training leads to a *poorly modeled outlier space*: as illustrated in the left side of Figure 2a, drift samples (representing a new malware family FakeInstaller) often become entangled with existing classes in the latent space, making it difficult to distinguish between them effectively.

(L2) Secondly, current detectors rely on training data to compute drift scores in operational phase. This dependence could be *impractical and inefficient*—it requires constant access to potentially large datasets, posing challenges in terms of security and scalability [59]. Moreover, we find that over-reliance on static training data risks overfitting, causing unseen samples from existing families to be misclassified as drift, further diminishing detection effectiveness.

3.2 Challenges in AL-based Adaptation

Active learning is employed to dynamically update classifier predictions in drift adaptation, involving human analysts in the process. To ensure the updated model is accurate, analysts assign new labels to a contextually determined number of drift samples (*labeling budget*), which are then incorporated into the original training data for retraining. This adaptation stage presents two challenges regarding the effectiveness and efficiency of human efforts.

(C1) One major challenge is minimizing the labeling budget needed to achieve target accuracy in the updated model. The common retraining approach is inherently inefficient, as it relies solely on the final family label provided by analysts, overlooking the valuable reasoning insights they apply during analysis. For example, analysts often derive their understanding about malware family from observing specific malicious behaviors through static and dynamic analysis [2], which offer valuable context beyond a simple label. The *failure to incorporate rich behavioral explanations* limits potential improvements in model updating. Consequently, this inefficiency increases the labeling burden, requiring more samples to be precisely analyzed [56, 73].

(C2) During label assignment, reducing expert effort in malware analysis remains a challenge [9, 46]. Current drift explanations fall short as generated at the feature level, where intelligibility issues exist especially for malware classifiers [30]. More importantly, unlike

explanations for non-drift data [27], drift explanations emphasize differences relative to a closest family. Given that features are static properties, such explanations provide *no additional insights beyond a simple diff calculation*.

3.3 Research Scope

In this research, we employ active learning to mitigate Android malware concept drift, as depicted in Figure 1. Cybersecurity teams deploy the malware classifier in a production environment online, while another system periodically evaluates its ability to make accurate decisions on incoming samples and take appropriate actions. This system consists of three key components. The detector (1) provides drift scores to indicate how much samples deviate from the existing distribution of labeled data. Detected samples, ranked and selected within a labeling budget, are then presented to malware analysts for review. The explainer (2) links detection decisions to semantically meaningful behaviors (features in existing research [78]), helping analysts understand the rationale behind the drift. Once the samples are labeled, the adaptor (3) uses them to expand the original training data and update the classifier.

Recognizing that intra-class drift scenarios have been more extensively explored, our work primarily addresses the challenges associated with inter-class drifts, specifically those caused by previously unseen malware families. Our goal is to enhance detection and adaptation strategies for such drifts, enabling accurate malware classification in the dynamic environment. By accurately identifying malware families, cybersecurity teams can take more targeted incident responses. Nevertheless, our proposed solution is not limited to inter-class drifts, as its generalizability to intra-class drifts will also be evaluated in Section 6.

4 Our DREAM System

We propose a system, named DREAM, for drift detection and drift adaptation within the active learning framework. In this section, we introduce design insights and technical details of the system.

4.1 Design Insights

To overcome the identified limitations and challenges, our core idea is to establish a *bidirectional connection* between the classifier and the drift detector, while embedding high-level *malware explanations* directly into the learning process. As shown in Figure 3, we design the drift detector using an autoencoder structure that incorporates

two key features: 1) aligning reconstructed samples with original samples in classifier predictions and 2) embedding malware behavioral concepts in latent representations. These features enable the drift detector to be informed by the *classifier knowledge* while also allowing *expert insights* to guide the classifier during adaptation, respectively addressing **L1** and **C1**, which directly impacts the accuracy of the updated classifier.

Specifically, we employ what we call *model-sensitive concept learning* to train the drift detector. As shown in Figure 2, the embedding space of the drift detector effectively separates different malware families with larger centroid distances, while drift samples are mapped to a clear outlier space. Furthermore, the alignment between the detector and classifier is evidenced by the classifier’s activation embeddings: original and reconstructed samples from the same non-drift family cluster together, creating a well-defined decision boundary that accurately distinguishes drift samples. This consistency also suggests that the learned behavioral concepts can, in turn, enhance classifier retraining during adaptation.

Following these designs, we achieve data autonomy (solving **L2**) by relying on reconstructed samples to calculate drifting scores, *eliminating the need for training data* during drift detection. To facilitate analysis on drift samples (addressing **C2**) in adaptation, we adjust the drift explainer to generate explanations within the concept space, providing *extended semantics beyond features* that align with human insights.

Key Notations. Let $\mathcal{X} \subseteq \mathbb{R}^{p \times q}$ denote the input feature space and $\mathcal{Y}$ the output label space. The classifier is represented as $\mathbf{M} : \mathcal{X} \rightarrow \mathcal{P}(\mathcal{Y})$, mapping features to label distributions. The latent embedding space is $\mathcal{Z} \subseteq \mathbb{R}^N$, learned via an autoencoder f , where an encoder f_{enc} maps $\mathcal{X} \rightarrow \mathcal{Z}$ and a decoder f_{dec} reconstructs $\mathcal{Z} \rightarrow \mathcal{X}$. We use $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{test}}$ to refer to the training and test datasets, respectively. In CADE, the autoencoder is trained with a reconstruction loss $\mathcal{L}_{\text{rec}}$ and a novel contrastive loss $\mathcal{L}_{\text{sep}}$ to enforce separation between samples from different families. In DREAM, we enrich the latent space $\mathcal{Z}$ into a malware concept space composed of an explicit concept space $\mathcal{Z}_{\text{exp}}$ and an implicit concept space $\mathcal{Z}_{\text{imp}}$. Built on this, we introduce a concept presence loss $\mathcal{L}_{\text{pre}}$ to align embeddings with behavioral labels, and a reliability loss $\mathcal{L}_{\text{rel}}$ to inject classifier knowledge for model-sensitive alignment. We elaborate on their roles in enhancing the drift detection and adaptation process in the following sections.

4.2 Concept-based Drift Detection

Model Sensitive Concept Learning. We explore the idea of concept learning to enhance the latent representations of the contrastive autoencoder. In machine learning, concept learning typically involves the task of inferring generalizable patterns (often framed as boolean-valued functions) from labeled examples to represent higher-level abstractions [17, 35]. In our context, we envision that learning generalized malicious behavior concepts can guide the model in better identifying new malware families under drift. To this end, our goal is to integrate the supervised concept learning with the unsupervised contrastive learning.

We define the unique latent space $\mathcal{Z}$ in our method as the malware concept space, comprising the explicit concept space and the implicit concept space. The explicit concept space $\mathcal{Z}_{\text{exp}}$ is structured

around a set of manually defined behaviors, where each element $\mathbf{z}_e^{(i)}$ corresponds to a predefined malicious concept (e.g., information stealing, remote control, and stealthy download). This mirrors the classic concept learning, where boolean-valued functions are used to assign interpretable concepts to samples. In contrast, the implicit concept space $\mathcal{Z}_{\text{imp}}$ captures statistical patterns from the data and serves as an automatically learned latent representation. It complements the explicit space by enriching the contrastive autoencoder and refining the contrastive loss $\mathcal{L}_{\text{sep}}$ in two ways: 1) improving class separation when explicit behaviors overlap across malware families, and 2) enabling concept space extensibility, such as accommodating benign behaviors in binary detection tasks.

Built on the malware concept space, we introduce two key training requirements focused on concept handling. Firstly, we implement a concept presence loss for the precise detection of explicit concepts for each sample, which is defined as

$$\mathcal{L}_{\text{pre}} = -\mathbf{m}_e \odot (\mathbf{l}_e \odot \log(\mathbf{p}_e) + (1 - \mathbf{l}_e) \odot \log(1 - \mathbf{p}_e)). \quad (1)$$

Here, $\odot$ denotes element-wise multiplication. The vectors $\mathbf{m}_e$, $\mathbf{l}_e$, and $\mathbf{p}_e$ each have a length of N_e , representing the total number of explicit concepts. The elements $\mathbf{m}_e^{(i)}$ and $\mathbf{l}_e^{(i)}$ correspond to the valid label mask and the binary label for the i -th concept, respectively; and $\mathbf{p}_e^{(i)}$ is calculated with $g(\mathbf{z}_e^{(i)})$, which is the probability of the i -th explicit concept being present. This formula aggregates binary cross-entropy for all explicit concepts, incorporating the valid label mask $\mathbf{m}_e$ to handle imprecise behavior labels. This strategy addresses the challenge of behavior labeling in the malware domain, where unlike the image domain with direct human annotations, concept labels often originate from technical reports or dynamic analysis and can be uncertain or missing [63].

Our second innovation is the introduction of the concept reliability loss, endowing the detector with model sensitivity from the early training stage. Our approach is based on the premise that a sample reconstructed from the concept space $\mathcal{Z}_{\text{exp}} \cup \mathcal{Z}_{\text{imp}}$ should exhibit a probability distribution similar to the original sample when processed by the classifier $\mathbf{M}$. The concept reliability loss is formally represented as

$$\mathcal{L}_{\text{rel}} = - \sum_i^C \mathbf{M}(\mathbf{x})^{(i)} \log(\mathbf{M}(\hat{\mathbf{x}})^{(i)}). \quad (2)$$

In this equation, the probability distributions of the original sample $\mathbf{x}$ serve as “true labels”, and those of the reconstructed sample $\hat{\mathbf{x}}$ are treated as predictions. This loss function effectively measures the divergence between the original and reconstructed sample distributions, ensuring that the model retains fidelity to the original concept representations even after reconstruction. Moreover, it implicitly measures the entropy-based uncertainty of the original distribution.

In summary, the training objective for our drift detector is centered on tuning the parameters of the autoencoder f and the concept presence function g , minimizing

$$\mathcal{L}(\mathcal{D}_{\text{train}}; \mathbf{l}, \mathbf{M}) = \lambda_0 \mathcal{L}_{\text{rec}} + \lambda_1 \mathcal{L}_{\text{sep}} + \lambda_2 \mathcal{L}_{\text{pre}} + \lambda_3 \mathcal{L}_{\text{rel}}, \quad (3)$$

where the traditional reconstruction loss, the concept-based contrastive loss, the concept presence loss, and the concept reliability loss are balanced by coefficients λ_0 , λ_1 , λ_2 , and λ_3 .

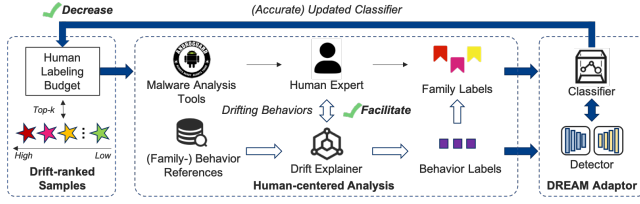


Figure 4: Human-in-the-loop strategies in DREAM. Bold blue arrows emphasize two core flows: (solid) system-level adaptation for efficient retraining with low budget and (hollow) human-centered behavioral explanation to assist labeling.

Data Autonomous Detector. With the trained detector, we can leverage it to detect drifting samples for incoming test data $\mathbf{x}_t \in \mathcal{D}_{\text{test}}$. Our design draws inspiration from a notable observation in the state-of-the-art intra-class detector HCC (Appendix B). Considering the significant contribution of the cross-entropy-based pseudo loss to the performance, we focus on enhancing its contrastive-based element. We achieve this by maintaining the neighborhood function while substituting the contrastive-based loss with the cross-entropy-based one, leading to the introduction of the neighborhood cross-entropy (NCE) based pseudo loss, defined as

$$\mathcal{D}_{\text{nce}}(\mathbf{x}_t; \mathcal{D}_{\text{train}}) = \hat{\mathcal{L}}_{\text{ce}}(\{\mathbf{x}_t\} \cup \mathcal{N}(\mathbf{x}_t; f, \mathcal{D}_{\text{train}})). \quad (4)$$

We also observe through a case study that this adjustment can improve HCC in its intra-class detection AUC by 6.11%.

To comprehend the underlying principle, it's important to understand that NCE calculates uncertainty for nearest neighbors based on the assumption that a non-drifting sample should remain certain even if slightly perturbed. This leads us to the samples reconstructed by the autoencoder, which essentially represent a constrained perturbation. Consequently, the pseudo loss defined with our concept reliability loss (Equation 2) can align well with this principle. The drift scoring function of our *Concept Reliability Detector* (CRD) is

$$u_d := \mathcal{D}_{\text{CRD}}(\mathbf{x}_t) = \hat{\mathcal{L}}_{\text{ce}}(\mathbf{x}_t; \mathbf{M}) + \lambda_3 \hat{\mathcal{L}}_{\text{rel}}(\mathbf{x}_t, f(\mathbf{x}_t)), \quad (5)$$

where the first item is the pseudo cross-entropy loss of the test sample in terms of the original classifier, and the second can be interpreted as the deviation in uncertainty after meaningful perturbations. This approach offers advantages for two primary reasons: 1) the perturbation $\hat{\mathbf{x}}_t = f(\mathbf{x}_t)$ is confined within the meaningful concept space, and the loss calculation considers the entropy of both the original sample and the perturbation; 2) the testing phase of the detector is specifically designed for data autonomy, ensuring its efficiency and operational independence from the training data.

4.3 Explanatory Drift Adaptation

As shown in Figure 4, DREAM addresses the human-in-the-loop challenges with two strategies: 1) a human-centered explanation mechanism to facilitate behavior labeling that can operate on the classifier, and 2) a system-level behavior-involved adaptation algorithm to reduce the labeling budget required for maintaining classifier accuracy.

Explaining with Concept. To implement the drift explainer, CADE introduces two key components: a perturbation function

and a deviation function. Specifically, given a drifting sample $\mathbf{x}_d$, it selects a reference sample $\mathbf{x}_r$ from the training data by first identifying the closest class centroid y_r , and then choosing the training sample whose latent representation is nearest to this centroid. The perturbation process involves using the mask $\mathbf{m} \in \mathbb{R}^{p \times q}$, resulting in the perturbed sample

$$\mathbf{x}'_d := \mathbf{x}_d \odot (1 - \mathbf{m}) + \mathbf{x}_r \odot \mathbf{m}. \quad (6)$$

The deviation is then quantified by calculating the distance between the latent representation of the perturbed sample, $\mathbf{z}'_d$, and the nearest class centroid, represented as $d(\mathbf{z}'_d, \mathbf{c}_{y_r})$.

Motivated by this approach, our method focuses on generating concept-based explanations while leveraging the model-sensitive nature of our detector. First of all, in terms of the selection of the nearest class y_r , we directly use $\hat{y}$ which is the class predicted by the original classifier. To create concept-based explanations, we introduce a concept-space mask $\mathbf{m}_c \in \mathbb{R}^N$. This mask redefines the perturbation function within the concept space, while still allowing us to compute the original distance-based deviation using latent representations. Further enhancing the deviation function, we incorporate the model-sensitive drifting scoring function. Therefore, the primary component of the explainer's optimization function is

$$\begin{aligned} & d(\mathbf{z}'_d, \mathbf{c}_{\hat{y}}) + \alpha_u \left(\hat{\mathcal{L}}_{\text{ce}}(\mathbf{x}'_d) + \hat{\mathcal{L}}_{\text{rel}}(f(\mathbf{x}_r), \mathbf{x}'_d) \right), \\ & \text{s. t. } \mathbf{z}'_d := \mathbf{z}_d \odot (1 - \mathbf{m}_c) + \mathbf{z}_r \odot \mathbf{m}_c, \quad \mathbf{x}'_d := f_{\text{dec}}(\mathbf{z}'_d), \end{aligned} \quad (7)$$

where the drifting score is balanced by α_u and determined by the feature-space perturbed sample $\mathbf{x}'_d$ decoded from the perturbed concepts. Specifically in the pseudo concept reliability loss $\hat{\mathcal{L}}_{\text{rel}}$, the sample reconstructed from the reference input, i.e., $f(\mathbf{x}_r)$, acts as a proxy for the in-distribution output probabilities. Note that our method can still generate feature-space explanations if needed: $\mathbf{x}'_d$ is created with feature-space operations as in Equation 6, and the pseudo concept reliability loss would be $\hat{\mathcal{L}}_{\text{rel}}(\mathbf{x}'_d, f(\mathbf{x}'_d))$.

Concept-involved Adaptor. Besides the drift explainer, our detector can serve as an explainer for in-distribution data in relation to the classifier. This is facilitated by the development of the explicit concept space. In this context, the outputs from the concept presence function g are intrinsically linked to the malicious behaviors identified by the system. When dealing with OOD samples, it is possible that these explanations may be inaccurate due to the evolving nature of the concepts they are based on. Nevertheless, the architecture of our detector is designed to allow analysts to utilize the drift explainer as a tool and refine these behavioral concepts. This adaptable design is instrumental in updating and enhancing the classifier's performance in response to conceptual drifts.

We enable human analysts to provide feedback not only on the predicted labels of drifting samples but also on their behavioral explanations (predicted concepts). For instance, in malware classification tasks, analysts might encounter a drifting sample identified as belonging to the GhostCtrl family, exhibiting malicious behaviors like PrivacyStealing, SMSCALL, RemoteControl, and Ransom. Feedback in this scenario can be formatted as:

$$\begin{aligned} & \exists mal \quad (\text{Family}(mal, \text{GhostCtrl})) \wedge (\text{Behaviors}(mal) \Leftrightarrow \\ & \quad \text{PrivacyStealing}(mal) \wedge \text{SMSCALL}(mal) \\ & \quad \wedge \text{RemoteControl}(mal) \wedge \text{Ransom}(mal)). \end{aligned}$$

Utilizing the feedback provided, our drift adaptor operates by concurrently tuning the classifier and the detector. Let Θ represent the parameters of the classifier's model, and Ψ denote those of the detector. The optimization problem, which aims to minimize the combined loss functions of the classifier and the detector, is formally defined as

$$\begin{aligned} \min_{\{\Theta, \Psi\}} & \left(\mathcal{L}_{ce}(\Theta) + \mathcal{L}_{det}(\Psi) + \lambda_3 \mathcal{L}_{rel}(\Theta, \Psi) \right), \\ \text{s. t. } & \mathcal{L}_{det} = \lambda_0 \mathcal{L}_{rec}(\Psi) + \lambda_1 \mathcal{L}_{sep}(\Psi) + \lambda_2 \mathcal{L}_{pre}(\Psi). \end{aligned} \quad (8)$$

The label feedback takes effect on the malware classification loss $\mathcal{L}_{ce}$ and the explanation feedback influences the concept presence loss $\mathcal{L}_{pre}$. Contrasting with the detector's training process where Θ is fixed (Equation 5), during the adaptation phase, both Θ and Ψ are subject to influence the concept reliability loss $\mathcal{L}_{rel}$. To address the complexities of joint parameter updating, we've implemented a dynamic learning rate schedule for the detector. Specifically, when $\mathcal{L}_{det}$ indicates that concept stability falls below a certain threshold, we reduce the learning rate using a scaling factor η . This reduction is based on the principle that stable concepts require less aggressive updates, promoting smoother model convergence and reducing the risk of overfitting.

5 System Evaluation

This section presents a systematic evaluation of DREAM, specifically focusing on inter-class drift scenarios, which are central to our research. We analyze the drift detector (Section 5.2), the drift adaptor, and their joint effectiveness in updating the classifier (Section 5.3). We also evaluate the drift explainer (Section 5.4) and perform a computational and human effort analysis of the system (Section 5.5).

5.1 Experimental Setup

Dataset. We employ two malware datasets for malware family classification tasks, as illustrated in Table 2. Our first dataset is the well-known Drebin dataset [3]. To capture more recent trends in the evolving malware landscape, we utilize the Malradar dataset [71]. For each dataset, we select 8 families with the same criteria used in the CADE paper, each comprising at least 100 malware samples. The resulting datasets consist of 3,317 and 2,589 malware samples for Drebin and Malradar, respectively. Both datasets are well-labeled with respect to malware families. Malradar also provides behavioral labels derived from threat reports for each family, and we select 10 distinct behaviors to form its behavioral labels: privacy information stealing (b_0), abusing SMS/CALL (b_1), remote control (b_2), bank/financial stealing (b_3), ransom (b_4), abusing accessibility (b_5), privilege escalation (b_6), stealthy download (b_7), aggressive advertising (b_8), and premium service (b_9). To ensure consistency in behavioral analysis, we augment the Drebin dataset with these same behaviors. For more details about the datasets, please also refer to Appendix A.

Classifier. We use three deep learning-based malware classifiers, leveraging the features and models defined in previous works. These classifiers differ in data modality and model complexity. 1) DREBIN [3] classifier utilizes eight feature sets representing binary vectors of predefined patterns, such as required permissions and suspicious API calls. The underlying model is a MLP [25], configured with two hidden layers, sized at 100 and 30 neurons

Family	# sample	(#) behavior	Time
FakeInstaller	925	(5) b_0, b_1, b_7, b_8, b_9	2011-2012
DroidKungFu	667	(3) b_0, b_2, b_6	2011-2012
Plankton	625	(4) b_0, b_2, b_7, b_8	2011-2012
GingerMaster	339	(2) b_0, b_6	2011-2012
BaseBridge	330	(4) b_0, b_1, b_6, b_9	2010-2011
Iconosys	152	(2) b_0, b_8	2010-2011
Kmin	147	(1) b_0	2010-2012
FakeDoc	132	(1) b_4	2011-2012
RuMMS	796	(4) b_0, b_1, b_2, b_3	2016-2018
Xavier	589	(4) b_0, b_2, b_7, b_8	2016-2021
LIBSKIN	290	(4) $b_0, b_1, b_2, b_6, b_7, b_8$	2015-2021
HiddenAd	289	(2) b_0, b_8	2017-2021
MilkyDoor	210	(2) b_0, b_2	2016-2020
GhostClicker	182	(4) b_0, b_2, b_6, b_8	2016-2020
EventBot	124	(5) b_0, b_1, b_2, b_3, b_5	2020
GhostCtrl	109	(3) b_0, b_1, b_2	2016-2020

Table 2: Drebin (top) and Malradar (bottom) datasets. In the third column, the number of behaviors is in parentheses, followed by their indices.

respectively. 2) MAMADROID [47] classifier involves extracting API call pairs and abstracting them into package call pairs. It builds a Markov chain to model the transitions between packages, using the derived float vectors as features. Its MLP architecture includes hidden layers with dimensions of 1,000 and 200. 3) DAMD [50] classifier leverages the raw opcode sequences as features. The sequence representation utilizes an embedding technique with a vocabulary size of 218 tokens and an embedding dimension of 128. Its underlying model is a CNN tailored for this task, featuring two convolutional layers, each with 64 filters.

Hold-out Strategy. To evaluate inter-class drifts, where the drift labels are determined based on whether a malware family was unseen during training, we employ a commonly used hold-out strategy [63, 78]. To implement this, we first exclude samples of each malware family from the training set, reserving them solely for the testing phase. The remaining families are then divided into an 80:20 ratio for training and testing, adhering to a time-based separation criterion [69]. Since both datasets contain 8 families, this strategy trains 8 classifiers per dataset, matching the number of malware families.

5.2 Drift Detection Performance

Baseline and Metric. For assessing our drift detector's performance in inter-class scenarios, we take into account the vanilla probability-based detector (using the negated maximum softmax output) and the innovative detectors from key studies summarized in Table 1. The HCC detector is excluded from this evaluation for being incompatible with inter-class contexts. To ensure a fair comparison, both the CADE detector and our detector, which each leverage an autoencoder model, are configured to share the same architecture across all features. For the metric, we utilize the AUC calculated from the detector's drifting score output and the ground truth labels, which are determined by the held-out malware families during the training process.

Evaluation Results. Figure 5 and Figure 7 depict the drift detection performance of our detector and the three baselines on the

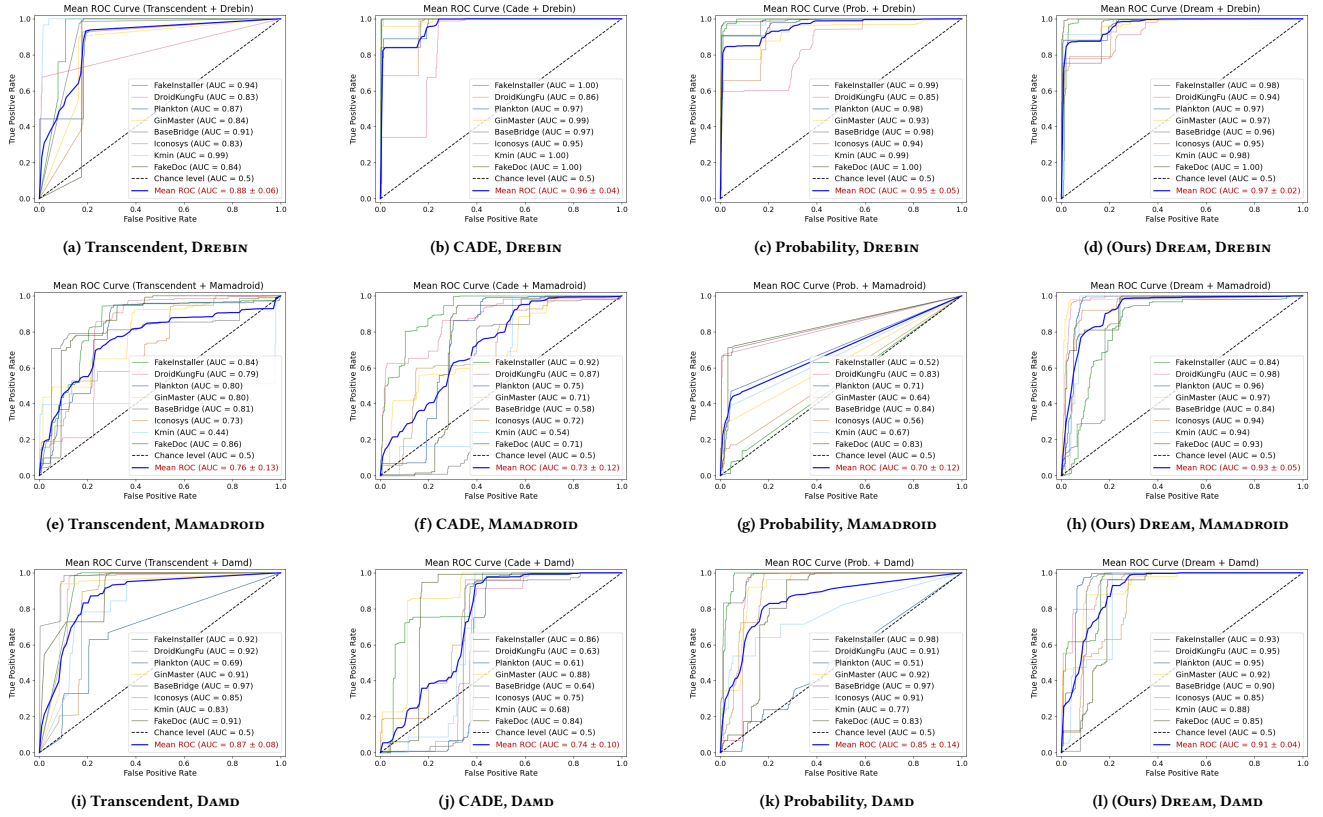


Figure 5: Evaluation of inter-class drift detection on Drebin dataset with three feature spaces. The first three columns are Transcendent, CADE, Probability, respectively, and our method is on the last column.

Drebin dataset and the Malradar dataset. Comparing the average AUC scores across different classifiers, we observe that DREAM outperforms Transcendent, CADE, and Probability by 11.95%, 15.64%, and 12.4%, respectively, on the Drebin dataset. Similarly, on the Malradar dataset, DREAM shows an increase of 10.98%, 8.33%, and 14.7%. In evaluating detection performance for different classifiers, we observed three key points. For the DREBIN classifier, which is simpler and where most methods excel, the CADE detector performs comparably to our method on both datasets. However, for classifiers with more complex feature spaces, such as the DAMD feature within the MalRadar dataset, CADE’s effectiveness decreases by 26.5% compared to the DREBIN feature, while our method shows a 22.2% advantage over CADE. Regarding the MAMADROID classifier, it is less accurate in training compared to other classifiers. For this classifier, model-sensitive baselines such as Transcendent and Probability perform poorly. Our method, however, demonstrates stability and benefits from concept-based contrastive learning. In the case of the DAMD classifier, the most complex among the tested, the Transcendent detector shows a smaller performance decline relative to other baselines. This is likely due to its calibration process, which boosts accuracy in scenarios of slight classifier overfitting (evidenced by a test accuracy of only 0.94, even 2.97% lower than that for the MAMADROID classifier). Although calibration proves beneficial, our method, with a focus on learning distance metrics,

is more efficient, showing an average improvement of 6.62% over the Transcendent detector.

5.3 Drift Adaptation Performance

Baseline and Metric. We compare our adaptation methods against commonly used techniques that retrain the classifier using selected samples and their annotated classification labels [15, 34, 78]. For comprehensive comparison and ablation analysis, we integrate the baseline adaptor with previous drift detectors. We categorize these four baseline adaptors into two groups, based on whether they use detectors that are data-autonomous or not. In particular, the Probability baseline adaptor serves as a benchmark in scenarios where drift detection must be performed locally without training data and without components from DREAM, allowing for a direct comparison under constraints similar to ours. Note that although the CRD baseline employs drift detector within our system, it uses traditional retraining without incorporating the concept revision component in our adaptor.

These experiments are conducted across a range of labeling budgets for active learning, specifically using absolute numbers of labeled samples—10, 20, 30, 40, and 100—with an emphasis on smaller budgets to minimize the need for extensive human annotation. The effectiveness of each approach is quantified by measuring

Feature	Budget	Metric	Drebin Dataset					MalRadard Dataset				
			Baseline Adaptor				DREAM Adaptor	Baseline Adaptor				DREAM Adaptor
			Trans.	CADE	Prob.	CRD		Trans.	CADE	Prob.	CRD	
DREBIN	10	F1	0.416	0.166	0.173	0.449	0.805 ↑93.3%	0.262	0.235	0.191	0.228	0.469 ↑78.9%
		Acc.	0.407	0.172	0.168	0.418	0.778 ↑91.4%	0.295	0.283	0.207	0.265	0.525 ↑78.0%
	20	F1	0.703	0.407	0.528	0.562	0.928 ↑31.9%	0.434	0.399	0.291	0.305	0.921 ↑112.3%
		Acc.	0.677	0.402	0.497	0.531	0.905 ↑33.7%	0.455	0.457	0.336	0.349	0.913 ↑100.0%
	30	F1	0.852	0.749	0.785	0.717	0.961 ↑12.7%	0.706	0.587	0.605	0.560	0.952 ↑34.9%
		Acc.	0.834	0.712	0.744	0.697	0.952 ↑14.1%	0.726	0.622	0.629	0.585	0.948 ↑30.7%
	40	F1	0.890	0.856	0.886	0.781	0.956 ↑7.42%	0.793	0.714	0.631	0.687	0.967 ↑21.9%
		Acc.	0.878	0.810	0.849	0.755	0.943 ↑7.40%	0.808	0.740	0.657	0.703	0.964 ↑19.3%
	100	F1	0.942	0.965	0.923	0.937	0.975 ↑1.10%	0.940	0.913	0.904	0.949	0.982 ↑4.45%
		Acc.	0.946	0.955	0.901	0.927	0.973 ↑1.87%	0.931	0.901	0.892	0.945	0.981 ↑5.43%
MAMA-DROID	10	F1	0.294	0.313	0.169	0.299	0.648 ↑106.9%	0.445	0.377	0.476	0.474	0.651 ↑46.3%
		Acc.	0.264	0.311	0.156	0.300	0.650 ↑108.6%	0.300	0.413	0.514	0.520	0.670 ↑36.9%
	20	F1	0.443	0.490	0.449	0.428	0.740 ↑51.0%	0.571	0.483	0.623	0.629	0.718 ↑25.8%
		Acc.	0.411	0.474	0.435	0.426	0.736 ↑55.2%	0.602	0.519	0.626	0.654	0.725 ↑20.4%
	30	F1	0.610	0.582	0.601	0.614	0.789 ↑29.4%	0.615	0.594	0.718	0.669	0.734 ↑19.5%
		Acc.	0.573	0.566	0.579	0.603	0.780 ↑36.1%	0.641	0.623	0.722	0.687	0.737 ↑14.9%
	40	F1	0.712	0.587	0.630	0.677	0.808 ↑13.5%	0.672	0.653	0.756	0.684	0.766 ↑14.0%
		Acc.	0.677	0.590	0.621	0.670	0.794 ↑17.2%	0.681	0.666	0.747	0.698	0.759 ↑11.4%
	100	F1	0.827	0.642	0.814	0.807	0.864 ↑4.47%	0.785	0.769	0.821	0.830	0.839 ↑6.88%
		Acc.	0.797	0.652	0.805	0.793	0.853 ↑6.97%	0.781	0.767	0.811	0.820	0.829 ↑6.12%
DAMD	10	F1	0.510	0.339	0.442	0.363	0.738 ↑44.6%	0.209	0.179	0.246	0.227	0.525 ↑151.2%
		Acc.	0.437	0.360	0.422	0.353	0.702 ↑60.6%	0.263	0.231	0.303	0.294	0.556 ↑111.1%
	20	F1	0.801	0.582	0.711	0.659	0.837 ↑4.50%	0.482	0.420	0.454	0.495	0.683 ↑41.7%
		Acc.	0.767	0.599	0.684	0.640	0.820 ↑6.98%	0.535	0.474	0.497	0.538	0.707 ↑32.2%
	30	F1	0.840	0.610	0.775	0.687	0.849 ↑1.07%	0.541	0.530	0.681	0.550	0.750 ↑38.5%
		Acc.	0.804	0.631	0.756	0.687	0.844 ↑4.96%	0.600	0.571	0.697	0.605	0.751 ↑25.3%
	40	F1	0.910	0.590	0.778	0.714	0.867 ↓4.74%	0.578	0.541	0.630	0.630	0.732 ↑26.7%
		Acc.	0.881	0.628	0.766	0.697	0.863 ↓2.05%	0.626	0.595	0.669	0.655	0.749 ↑19.7%
	100	F1	0.955	0.652	0.888	0.904	0.955 ↓0.01%	0.753	0.669	0.768	0.770	0.811 ↑7.64%
		Acc.	0.941	0.654	0.872	0.875	0.941 ↑0.03%	0.761	0.697	0.778	0.777	0.809 ↑6.29%

Table 3: Drift adaptation results on two malware datasets across three feature sets. We integrate the baseline adaptor with different drift detectors and categorize them into two groups based on their applicable scenarios: those that require access to the original training data for drift detection (Transcendent, CADE) and those that do not (Probability, CRD). Baselines with the best performance in each scenario are highlighted in bold, and our respective improvement ratios are reported in blue.

F1-scores and accuracy scores of the updated classifiers on the remaining test dataset. For inter-class adaptation, an important step is the modification of the classifier output to include new classes. This modification involves randomly initializing the new output layer while preserving the learned weights in the existing layers, enabling continuous utilization of prior knowledge, and is consistently applied across all methods.

Evaluation Results. Table 3 shows the drift adaptation results across different labeling budgets on all datasets and classifiers.

1) *Benchmark Comparison.* Our method outperforms the benchmark in different settings without exception. Firstly, regarding performance improvements with different classifiers, we observed significant enhancements. On the Drebin dataset, the improvements of F1-score are 95.2%, 82.7%, and 22.6% for DREBIN, MAMADROID, and DAMD classifiers, respectively. On the Malradar dataset, the improvements are 96.3%, 11.6%, and 39.2% for the same classifiers. Secondly, when considering varying human labeling budgets (10, 20, 30, 40, 100), the improvements of F1-score on the Drebin dataset are 238.1%, 52.8%, 21.1%, 15.8%, and 6.5%, respectively, while being 98.7%, 94.2%, 23.3%, 23.6%, and 5.5% on the Malradar dataset for

these respective budgets. A key finding is that the smaller the budget, the greater the improvement. This suggests that in scenarios prioritizing human analysis, our method can significantly reduce labeling budgets. For example, achieving an accuracy score of 0.9 on the DREBIN feature of the Drebin dataset requires labeling only 20 new samples with DREAM, compared to 100 samples with the benchmark, reducing analysis cost by 80%.

2) *Existing Work Comparison.* Compared with the two baselines that utilize data-dependent drift detectors from existing work, DREAM maintains a clear advantage across the two datasets. When evaluating across classifiers, DREAM achieves F1-score improvements over Transcendent of 40.1%, 34.7%, and 31.1%, respectively. The improvements over CADE are more substantial, at 89.0%, 43.7%, and 64.8%. Similarly, when comparing under varying annotation budgets, DREAM outperforms Transcendent by 89.1%, 47.2%, 22.7%, 13.1%, and 4.5%; compared with CADE, the improvements are 162.7%, 77.5%, 38.4%, 30.7%, and 20.0%. Comparing the two baselines, we observe an interesting phenomenon: the Transcendent-based adaptor outperforms the CADE-based in over 83% of cases, even if it identifies fewer true drift samples. For instance, within the

Explainer	Metric	f0	f1	f2	f3	f4	f5	f6	f7	Avg.
Random	CBP	0.239	0.145	0.131	0.070	0.093	0.191	0.225	0.057	0.144
	DRR	0.212	0.175	0.218	0.153	0.201	0.202	0.324	0.316	0.225
Dri-IG	CBP	0.154	0.217	0.166	0.246	0.103	0.194	0.299	0.208	0.198
	DRR	0.134	0.281	0.287	0.464	0.263	0.263	0.469	0.463	0.328
CADE ⁺	CBP	0.347	0.372	0.312	0.319	0.320	0.361	0.341	0.436	0.351
	DRR	0.834	0.722	0.781	0.791	0.788	0.752	0.808	0.819	0.787
DREAM	CBP	0.348	0.408	0.331	0.402	0.338	0.372	0.487	0.443	0.391
	DRR	0.835	0.733	0.797	0.800	0.792	0.765	0.842	0.819	0.798

Table 4: Explanation evaluation results on the Drebin dataset with CBP and DRR metrics. The columns labeled f0 to f7 represent the held-out families, arranged in descending order by the number of samples in each family within the dataset.

budget 10, even if Transcendent correctly detects only 2 samples on average compared to CADE’s 9.3, the updated model using Transcendent still achieves a 0.25 advantage in F1-score. This performance can be attributed to Transcendent’s focus on statistical decision boundaries rather than true malware patterns. As a result, it inaccurately detects new families but improves retraining performance, especially for complex classifiers like those on DAMD. Transcendent even achieves slightly higher F1-score than DREAM in two high-budget cases on this classifier within the Drebin dataset. For the two specific instances, we combine Transcendent with our adaptor and improve its performance by 1.7% and 1.4%. However, this combination requires training both our autoencoder and Transcendent’s model sets, making it computationally expensive and limited to data-dependent detection scenarios.

3) *Ablation Analysis.* Within the selected low budgets in adaptation, we observe that the Probability and CRD detectors are often comparable in accurately detecting samples from new families, outperforming existing detectors in 86.7% cases. Notably, our previous detection evaluation uses the AUC metric, showing that the CRD detector demonstrates an advantage in overall accuracy across different budgets. To clarify this difference, we threshold the number of samples that are truly drifting, finding that the CRD detector achieves an accuracy 14.56% higher than the Probability detector. When the two detectors are integrated with the baseline adaptor, their performance remains similar, each succeeding in half of the specific cases. However, when paired with DREAM adaptor, we find that the Probability falls 12.8% short of DREAM in terms of F1-score of the updated classifier. These findings suggest that the majority of performance gains in adaptation stem from our updating mechanism. While the detected samples are crucial, their impact is highly specific to our system, as the primary value lies in how DREAM utilizes them to update the classifier, with our detector incorporating behavioral concepts and actively participating in the process.

5.4 Drift Explanation Performance

Baseline and Metric. We consider three baseline methods for drift explanation, adapted to generate both concept-level and traditional feature-level explanations: 1) a random baseline that randomly selects features or concepts as important; 2) a gradient-based explainer that utilizes Integrated Gradients (IG) [66], adapted here to analyze drifting scores derived from our detector. Originally designed to attribute a deep network’s predictions to its input features, IG has demonstrated effectiveness in explicating supervised security applications [30, 72]. In this context, it is tailored to focus on the drifting

scores derived from model predictions, and the reference sample $\mathbf{x}_r$ serves as the baseline in its method; 3) the state-of-the-art CADE explainer used with the CADE detector, as described in Figure 4.3. Since this method is not inherently designed for concept-space explanations, we adapt it to our detector for generating explanations in the concept space.

To evaluate the effectiveness of the drift explainers, we design two metrics. The first metric, named Cross Boundary P-value (CBP), assesses if explanations enable samples to cross the decision boundary, a key aspect in evaluating eXplainable AI (XAI) methods [61]. The second metric, as suggested in existing research [78], focuses on the distance to the reference sample in the detector’s latent space after perturbation. We provide their details in Appendix C. For a balanced comparison, particularly against the first two baseline methods that only yield importance scores, we align the number of altered features or concepts with those pinpointed by our techniques. These experiments are conducted using the DREBIN feature of the Drebin dataset, maintaining consistency with the CADE paper, where the performance of its drift detector is established.

Evaluation Results. Table 4 illustrates the evaluation results of concept-based explanations generated by different methods. Comparing to three baseline explainers, i.e., Random, Dri-IG and CADE⁺, DREAM surpasses them by 172.1%, 97.2% and 11.5% on the CBP metric, and by 254.2%, 143.3% and 1.4% on the DRR metric. Notably, CADE⁺ is included in this comparison as it utilizes our detector for concept-based explanations, extending beyond its original method’s capabilities. To examine the efficacy in areas typically addressed by common methods, Table 8 presents the outcomes for feature-level explanation evaluations. Here, our method, despite not being primarily designed for feature-level explanations, outperforms all baselines, including the state-of-the-art CADE explainer, benefiting from our method’s sensitive capture of deviations. Combining the results from both tables, our explainer yields more substantial improvements in concept space than in feature space. For instance, while a 45.2% increase in mean CBP over Dri-IG is shown at the feature level, a remarkable 97.2% increase is illustrated at the concept level, highlighting the effectiveness of our design in concept space.

5.5 Computational & Human Effort Analysis

In this analysis, we investigate the computational cost involved in drift detection & adaptation and the human effort required to generate correct labels for accurate adaptation. All the following experiments are conducted using a single NVIDIA A6000 GPU, with results reported on the DREBIN features and the MalRadar dataset.

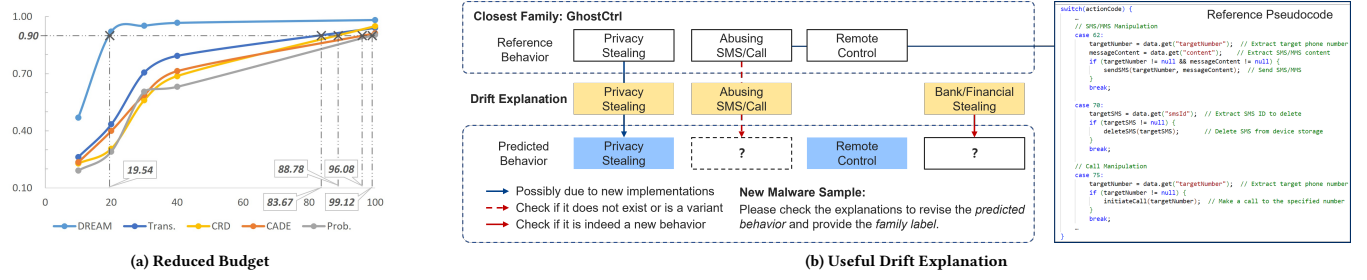


Figure 6: Human effort analysis: demonstration of how DREAM can (a) reduce the labeling budget needed to maintain a certain accuracy level and (b) facilitate the labeling process through explainability.

Computational Performance Analysis. Regarding the drift detection investment, we consider both the training and testing phases of the detector. During training, our detector has similar computational overhead as CADE and remains more efficient than Transcend. Specifically, the training computational complexity follows CADE’s model, expressed as $O(IB^2|\Psi|)$, where I , B , and $|\Psi|$ represents the number of training iterations, batch size, and the number of model parameters. Although our detector is slightly larger than CADE’s due to an additional dense layer for concept presence prediction, the small number of high-level concepts (e.g., 10 malicious behaviors) keeps the model sizes comparable, both estimated at 5.58 MB; training time is slightly higher at 0.81 s per epoch compared to 0.74 s. For Transcendent, evaluated in its rather computationally friendly setting with 10-fold, has a model size of about 5.45 MB but a longer training epoch time of 1.12 s. During testing phase, where efficiency is important for real-time applications, DREAM demonstrates superior performance by requiring only 0.57 ms per sample without additional data. In contrast, CADE and Transcendent require 1.89 ms and 5.75 ms, respectively. Moreover, their dependence on training data increases memory usage.

In terms of operations for tuning the model, the concept-involved adaptor in DREAM incurs the most computational cost. The retraining time is 2.86 s per epoch, while the baseline adaptor takes 0.64 s. Despite this, the time remains acceptable—fine-tuning requires much fewer iterations than training from scratch (50 epochs in our experiments)—and is unlikely to be a barrier to deployment [15]. The primary focus here is to achieve higher accuracy in the adapted model and reduce human effort, as will be explained subsequently.

Human Effort Analysis. The total human effort can be approximated by $n \times w$, where n is the number of samples to be analyzed, and w is the analysis workload per sample. DREAM reduces human effort by decreasing 1) n : utilizing an effective drift adaptor that requires significantly fewer budgets to achieve high accuracy (Section 5.3), and 2) w : providing an effective explainer that pinpoints key concepts faithfully (Section 5.4). In the following, we further discuss the first aspect and support the second with a case study.

Firstly, as shown in Figure 6a, we present the budget versus accuracy curves derived from previous adaptation performance results. By selecting the desired classification accuracy threshold, we estimate the required labeling budget. Typically, to achieve 90% accuracy in the updated model, DREAM requires only 19.54 samples to be analyzed. In contrast, the baseline method requires 99.12 samples, and even when enhanced with more time-consuming

Detector	2016	2017	2018	2019	2020	Avg.
HCC ⁺	0.741 ↑4.8%	0.718 ↑10.7%	0.734 ↑21.4%	0.765 ↑6.3%	0.699 ↑6.8%	0.731 ↑9.7%
HCC ^{ce}	0.745 ↑4.8%	0.724 ↑11.5%	0.719 ↑18.7%	0.768 ↑6.8%	0.698 ↑6.6%	0.731 ↑9.4%
HCC ^{hc}	0.637 ↑9.6%	0.700 ↑15.0%	0.714 ↑20.6%	0.713 ↑5.2%	0.684 ↑4.1%	0.690 ↑10.6%
DREAM	0.754 ↑6.6%	0.781 ↑20.3%	0.866 ↑43.1%	0.791 ↑10.0%	0.763 ↑16.6%	0.791 ↑18.6%

Table 5: The enhancement of intra-class drift detection with our detection framework.

detectors, the best alternative still requires 83.67 samples. This substantial reduction translates to 80.3% savings in n .

Secondly, to understand DREAM’s explanation impact, we present a case study using drift explanations for a sample from RuMMS (the largest family in the dataset), which mainly targets banking information through SMS interception [22]. As illustrated in Figure 6b, DREAM detects GhostCtrl as the closest family in the training data. While this family may serve as a reference, it focuses on remote control functionalities for data theft and manipulation [41], and our drift explainer identifies 3 drifting behaviors out of the 10 predefined concepts. This explanation was manually verified to be useful in assigning the correct family label. For instance, the identification of *bank stealing* as a new behavior allowed analysts to effectively prioritize their investigation and find the primary malicious intent of the RuMMS malware. Furthermore, reviewing the reference pseudocode revealed that *abusing phone calls* is also a significant drift: while GhostCtrl exhibited diverse and aggressive SMS/CALL functionalities, RuMMS specialized in SMS interception for data exfiltration. This specialization enabled analysts to understand the sample’s more focused but narrower abusing behaviors.

6 Extended Evaluation

To broaden our focus, we examine DREAM’s effectiveness in intra-class drift. In Appendix B, we evaluate drift detection methods from related work and demonstrate HCC’s best overall performance on the dataset for binary malware detection. Adopting the same experimental setup, we compare DREAM against HCC in intra-class drift detection and adaptation.

Intra-class Drift Detection. DREAM achieves an 18.57% AUC improvement on average compared to the HCC detector. We further enhance the baseline with our insights about model sensitivity

Budget	Adaptor	2016	2017	2018	2019	2020
10	HCC	0.9199	0.9151	0.8948	0.9070	0.8691
	DREAM	0.9206	0.9191	0.8959	0.9074	0.8708
20	HCC	0.9219	0.9204	0.8971	0.9088	0.8753
	DREAM	0.9228	0.9239	0.8982	0.9142	0.8733
30	HCC	0.9221	0.9221	0.8990	0.9090	0.8903
	DREAM	0.9196	0.9226	0.8998	0.9109	0.8870
40	HCC	0.9246	0.9217	0.8994	0.9128	0.8892
	DREAM	0.9208	0.9231	0.9001	0.9112	0.8857
100	HCC	0.9241	0.9239	0.9017	0.9157	0.8890
	DREAM	0.9291	0.9304	0.9018	0.9176	0.8971

Table 6: F1-score comparison of HCC and DREAM in intra-class drift adaptation across different labeling budget.

and the NCE metric, achieved by 1) the removal of the surrogate classifier M_s and its integration into the original classifier using the autoencoder structure; 2) the augmentation of the previous cross-entropy based pseudo loss item with NCE (Equation 4). As presented in Table 5, our modification achieves significant enhancements in HCC’s detection performance, evident by both the integrated method and its two individual pseudo-loss detectors. Notably, there is an average performance increase from 9.43% to 10.62%. The contrastive-based pseudo-loss detector, previously the least effective, now surpasses all prior baselines with the most substantial improvement. Nevertheless, our approach maintains an 8.21% lead against the improved HCC method, largely due to our enriched latent space and data-autonomous design.

Intra-class Drift Adaptation. We integrate the HCC detector with the baseline adaptor to enable updates to external classifiers. As presented in Table 6, DREAM exceeds the HCC baseline in 76% of cases, with more improvements observed at the higher labeling budget (i.e., 4.73% at budget 100). However, the gains are less pronounced compared to prior intra-class drift detection or our adaptor’s performance in inter-class scenarios, which can be attributed to two main reasons. First, DREAM detects more malware samples than the baseline (51% vs. 40%). Due to the highly imbalanced nature of binary malware detection (10 : 1 goodware to malware [55]), malware samples, despite being more prone to drift, represent a smaller portion of the retraining process. Second, our adaptor’s design emphasizes embedding malware concepts to differentiate families. When adapting to a binary detection task, all benign samples are labeled with zero concept vectors. Given the larger proportion of benign samples, this limits our semantic enrichment, resulting in less effective adaptation.

7 Discussion

Concepts and Annotators. We define malicious Android behaviors as concepts to effectively handle inter-class drift. These concepts can be extended to cover benign functionality for intra-class drift and platform-specific behaviors beyond Android. For example, PE (Windows) malware often exhibits behaviors such as process injection, registry persistence, and credential access [63]. While some rootkit-like PE malware may involve deeper system-level integration, these behaviors still fall into structured categories (e.g., privilege escalation), similar to those in Android malware,

so the concept space is not necessarily larger. To support generalization, we envision leveraging standardized taxonomies such as MITRE ATT&CK [65], which captures cross-platform adversarial techniques organized hierarchically, enabling both coarse- and fine-grained modeling of explicit concepts. To reduce labeling overhead, concept annotation can be supported by automated analysis tools [19, 45] or LLM-based methods [6, 10], enhancing applicability across diverse malware triage settings.

Robustness to Attacks. Autoencoders have shown promise in anomaly detection [28] and have been extended to detect attacks. For instance, CAE [58] applies a classifier in the autoencoder’s latent space to identify poisoning samples, and MagNet [51] trains multiple autoencoders independently of the target classifier to detect adversarial attacks. Interestingly, the autoencoder-based drift detector CADE [78] is proven capable of detecting adversarial malware samples [39], and active learning is also promising in mitigating poisoning attacks [44, 49]. Although attack defense is beyond our research scope, DREAM combines autoencoder-based drift detection with active learning, incorporating both classifier and expert knowledge, potentially enhancing robustness against such attacks.

8 Related Work

OOD Detection. The majority of existing OOD detection methods in machine learning community rely on auxiliary OOD dataset [12, 31, 43, 48]. For instance, Chen et al. [12] uses an auxiliary dataset like the 80 Million Tiny Images. However, acquiring such large-scale and comprehensive auxiliary OOD datasets can be particularly challenging in the malware domain. Similar to existing work in malware domain, we are not dependent on auxiliary OOD dataset to offer greater practicality. Moreover, DREAM operates independently of any training data during the drift detection phase, enhancing its applicability in real-world scenarios.

Labelless Drift Adaptation. Besides active learning, there are also drift adaptation strategies without labels [38]. For example, APIGraph [80] and AMDASE [76] use semantically-equivalent API usages to mitigate classifier aging [75]. However, they only apply to classifiers with specific types of features. DroidEvolver [38, 75] employs pseudo-labels and is a promising solution to address labeling capacity. Nevertheless, this method can easily lead to negative feedback loops and self-poisoning [7]. Our work is based on active learning which involves human and effectively minimizes the labeling budget. Nevertheless, these works can be complementary to us to further enhance the robustness in drift adaptation.

Explainable Security Applications. Recent research has focused on offering post-hoc explanations to security applications. For example, in malware detection [1, 3] tasks, FINER [30] produces function-level explanations to facilitate code analysis. In malware mutation [13, 40] applications, AIRS [79] explains deep reinforcement learning models in security by offering step-level explanations. On a different basis, DREAM addresses the explainability problem in a drift adaptation setting, where the intrinsic behavioral explanations can propagate expert revisions to update the classifier.

Explanatory Interactive Learning. Recent advancements in machine learning have combined explainable AI with active learning, leading to progress in explanatory interactive learning [60, 62, 64]. Inspired by this, our approach incorporates human feedback on both

labels and explanations, but with distinct objectives and settings. Unlike these works that focus on image domain and feature-level explanation annotation, our method is tailored for malware analysis and generates high-level explanations. Furthermore, while they use external explainers to guide classifiers [64], our approach integrates explanations directly within the drift detector, enhancing the efficiency of our drift adaptation method.

9 Conclusion

To deploy deep learning-based malware classifiers in dynamic and hostile environments, our work addresses a crucial aspect of combating concept drift. The proposed DREAM system emerges as an innovative and effective solution. Behavioral explanations are integrated into a contrastive autoencoder, connecting the classifier to achieve model-sensitive detection and explanatory adaptation. The effectiveness of DREAM against evolving threats is demonstrated through extensive evaluation and marks a notable advancement over existing methods. We make the system open-source² and hope that it can inspire future research to explore concept drift in broader security contexts.

Acknowledgments

We would like to thank the anonymous reviewers for their helpful comments. This work was supported in part by the National Natural Science Foundation of China under Grant 62441238.

References

- [1] Hyrum S Anderson and Phil Roth. 2018. Ember: an open dataset for training static pe malware machine learning models. *arXiv preprint arXiv:1804.04637* (2018).
- [2] Simone Aonzo, Yufei Han, Alessandro Mantovani, and Davide Balzarotti. 2023. Humans vs. machines in malware classification. In *32nd USENIX Security Symposium (USENIX Security 23)*. 1145–1162.
- [3] Daniel Arp, Michael Spreitzenbarth, Malte Hubner, Hugo Gascon, Konrad Rieck, and CERT Siemens. 2014. Drebin: Effective and explainable detection of android malware in your pocket. In *Proceedings of the Network and Distributed Systems Security Symposium (NDSS)*, Vol. 14. 23–26.
- [4] AV-ATLAS. 2023. New malware. <https://portal.av-atlas.org/>.
- [5] Erin Avilazagaj, Ziyun Zhu, Leyla Bilge, Davide Balzarotti, and Tudor Dumitras. 2021. When malware changed its mind: An empirical study of variable program behaviors in the real world. In *30th USENIX security symposium (USENIX Security 21)*. 3487–3504.
- [6] Parikshit Bansal and Amit Sharma. 2023. Large language models as annotators: Enhancing generalization of nlp models at minimal cost. *arXiv preprint arXiv:2306.15766* (2023).
- [7] Federico Barbero, Feargus Pendlebury, Fabio Pierazzi, and Lorenzo Cavallaro. 2022. Transcending transcend: Revisiting malware classification in the presence of concept drift. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 805–823.
- [8] Jean Paul Barddal, Heitor Murilo Gomes, Fabricio Enembreck, and Bernhard Pfahringer. 2017. A survey on feature drift adaptation: Definition, benchmark, challenges and future directions. *Journal of Systems and Software* 127 (2017), 278–294.
- [9] Kevin Burk, Fabio Pagani, Christopher Kruegel, and Giovanni Vigna. 2022. Decomperson: How Humans Decompile and What We Can Learn From It. In *31st USENIX Security Symposium (USENIX Security 22)*. 2765–2782.
- [10] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. 2023. Large language models as tool makers. *arXiv preprint arXiv:2305.17126* (2023).
- [11] Fabricio Ceschin, Marcus Botacin, Heitor Murilo Gomes, Felipe Pinagó, Luiz S Oliveira, and André Grégio. 2023. Fast & Furious: On the modelling of malware detection as an evolving data stream. *Expert Systems with Applications* 212 (2023), 118590.
- [12] Jiefeng Chen, Yixuan Li, Xi Wu, Yingyu Liang, and Somesh Jha. 2020. Robust out-of-distribution detection for neural networks. *arXiv preprint arXiv:2003.09711* (2020).
- [13] Kai Chen, Peng Wang, Yeonjoon Lee, Xiaofeng Wang, Nan Zhang, Heqing Huang, Wei Zou, and Peng Liu. 2015. Finding unknown malice in 10 seconds: Mass vetting for new threats at the [Google-Play] scale. In *24th USENIX Security Symposium (USENIX Security 15)*. 659–674.
- [14] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. 2020. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*. PMLR, 1597–1607.
- [15] Yizheng Chen, Zhoujie Ding, and David Wagner. 2023. Continuous learning for android malware detection. In *32nd USENIX Security Symposium (USENIX Security 23)*. 1127–1144.
- [16] Zhuo Chen, Jie Liu, Yubo Hu, Lei Wu, Yajin Zhou, Yiling He, Xianhao Liao, Ke Wang, Jinku Li, and Zhan Qin. 2023. Deuedroid: Detecting underground economy apps based on utg similarity. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 223–235.
- [17] Thomas KF Chiu and Daniel Churchill. 2016. Design of learning objects for concept learning: Effects of multimedia learning principles and an instructional approach. *Interactive Learning Environments* 24, 6 (2016), 1355–1370.
- [18] Theo Chow, Zeliang Kan, Lorenz Linhardt, Daniel Arp, Lorenzo Cavallaro, and Fabio Pierazzi. 2023. Drift Forensics of Malware Classifiers. In *Proc. of the ACM Workshop on Artificial Intelligence and Security (AISec)*. ACM.
- [19] Manuel Egele, Theodor Scholte, Engin Kirda, and Christopher Kruegel. 2008. A survey on automated dynamic malware-analysis techniques and tools. *ACM computing surveys (CSUR)* 44, 2 (2008), 1–42.
- [20] Jerome Fan, Suneel Upadhye, and Andrew Worster. 2006. Understanding receiver operating characteristic (ROC) curves. *Canadian Journal of Emergency Medicine* 8, 1 (2006), 19–20.
- [21] Ming Fan, Jun Liu, Xiapu Luo, Kai Chen, Tianyi Chen, Zhenzhou Tian, Xiaodong Zhang, Qinghua Zheng, and Ting Liu. 2016. Frequent subgraph based familial classification of android malware. In *2016 IEEE 27th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 24–35.
- [22] Wu Zhuo (Fireeye). 2016. RuMMS: The Latest Family of Android Malware Attacking Users in Russia Via SMS Phishing. <https://vulners.com/fireeye/FIREEYE:9AE5925A39D370A4CBCA111D177A7AB9>.
- [23] Ekta Gandotra, Divya Bansal, and Sanjeev Sofat. 2014. Malware analysis and classification: A survey. *Journal of Information Security* 2014 (2014).
- [24] Liangyi Gong, Zhenhua Li, Feng Qian, Zifan Zhang, Qi Alfred Chen, Zhiyun Qian, Hao Lin, and Yunhao Liu. 2020. Experiences of landing machine learning onto market-scale mobile malware detection. In *Proceedings of the Fifteenth European Conference on Computer Systems*. 1–14.
- [25] Kathrin Grosse, Nicolas Papernot, Praveen Manoharan, Michael Backes, and Patrick McDaniel. 2017. Adversarial examples for detection. In *Computer Security—ESORICS 2017: 22nd European Symposium on Research in Computer Security, Oslo, Norway, September 11–15, 2017, Proceedings, Part II 22*. Springer, 62–79.
- [26] Alejandro Guerra-Manzanares, Marcin Luckner, and Hayretin Bahsi. 2022. Concept drift and cross-device behavior: Challenges and implications for effective android malware detection. *Computers & Security* 120 (2022), 102757.
- [27] Wenbo Guo, Dongliang Mu, Jun Xu, Purui Su, Gang Wang, and Xinyu Xing. 2018. Lemna: Explaining deep learning based security applications. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 364–379.
- [28] Dongqi Han, Zhiliang Wang, Wenqi Chen, Kai Wang, Rui Yu, Su Wang, Han Zhang, Zhihua Wang, Minghui Jin, Jiahai Yang, et al. 2023. Anomaly Detection in the Open World: Normality Shift Detection, Explanation, and Adaptation. In *Proceedings of the Network and Distributed Systems Security Symposium (NDSS)*.
- [29] Yiling He, Yiping Liu, Lei Wu, Ziqi Yang, Kui Ren, and Zhan Qin. 2023. MsDroid: Identifying Malicious Snippets for Android Malware Detection. *IEEE Transactions on Dependable and Secure Computing* 20, 3 (2023), 2025–2039.
- [30] Yiling He, Jian Lou, Zhan Qin, and Kui Ren. 2023. FINER: Enhancing State-of-the-art Classifiers with Feature Attribution to Facilitate Security Analysis. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 416–430.
- [31] Dan Hendrycks and Kevin Gimpel. 2016. A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks. In *International Conference on Learning Representations*.
- [32] Wei-Ning Hsu and Hsuan-Tien Lin. 2015. Active learning by learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 29.
- [33] Asif Iqbal, Subodh Kant Tiwari, Sikander Azad, Mithun Kumar Paswan, et al. 2024. Android Based Malware Detection Technique Using Machine Learning Algorithms. In *2024 First International Conference on Pioneering Developments in Computer Science & Digital Technologies (IC2SDT)*. IEEE, 1–6.
- [34] Steve TK Jan, Qingying Hao, Tianrui Hu, Jiameng Pu, Sonal Oswal, Gang Wang, and Bimal Viswanath. 2020. Throwing darts in the dark? detecting bots with limited data using neural data augmentation. In *2020 IEEE symposium on security and privacy (SP)*. IEEE, 1190–1206.
- [35] Yangqing Jia, Joshua T Abbott, Joseph L Austerweil, Tom Griffiths, and Trevor Darrell. 2013. Visual concept learning: Combining machine vision and bayesian generalization on concept hierarchies. *Advances in Neural Information Processing Systems* 26 (2013).
- [36] Yu Jiang, Ruixuan Li, Junwei Tang, Ali Davanian, and Heng Yin. 2020. Aomdroid: detecting obfuscation variants of android malware using transfer learning. In *Security and Privacy in Communication Networks: 16th EAI International Conference, SecureComm 2020, Washington, DC, USA, October 21–23, 2020, Proceedings, Part II 16*. Springer, 242–253.
- [37] Roberto Jordane, Kumar Sharad, Santanu K Dash, Zhi Wang, Davide Papini, Ilia Nouretdinov, and Lorenzo Cavallaro. 2017. Transcend: Detecting concept drift in malware classification models. In *26th USENIX security symposium (USENIX security 17)*. 625–642.
- [38] Zeliang Kan, Feargus Pendlebury, Fabio Pierazzi, and Lorenzo Cavallaro. 2021. Investigating labelless drift adaptation for malware detection. In *Proceedings of the 14th ACM Workshop on Artificial Intelligence and Security*. 123–134.
- [39] KASTEL: Cryptography and Security Group. 2021. *Detecting Adversarial Malware Examples as Concept Drift*. https://crypto.iti.kit.edu/english/seminar_ba_detecting_adversarial_malware_examples_as_concept_drift.php Accessed on November, 2023.
- [40] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. 2017. Simple and scalable predictive uncertainty estimation using deep ensembles. *Advances in neural information processing systems* 30 (2017).
- [41] Cedric Pernet (TrendMicro) Lenart Bermejo, Jordan Pan. 2017. Android Backdoor GhostCtrl Records Your Audio, Video. https://www.trendmicro.com/en_us/research/17/g/android-backdoor-ghostctrl-can-silently-record-your-audio-video-and-more.html.
- [42] Heng Li, Shiyao Zhou, Wei Yuan, Xiapu Luo, Cuiying Gao, and Shuiyan Chen. 2021. Robust android malware detection against adversarial example attacks. In *Proceedings of the Web Conference 2021*. 3603–3612.
- [43] Shiyu Liang, Yixuan Li, and Rayadurgam Srikant. 2017. Enhancing the reliability of out-of-distribution image detection in neural networks. *arXiv preprint arXiv:1706.02690* (2017).
- [44] Jing Lin, Ryan Luley, and Kaiqi Xiong. 2021. Active learning under malicious mislabeling and poisoning attacks. In *2021 IEEE Global Communications Conference (GLOBECOM)*. IEEE, 1–6.
- [45] Davide Lorenzoli, Leonardo Mariani, and Mauro Pezzè. 2008. Automatic generation of software behavioral models. In *Proceedings of the 30th international conference on Software engineering*. 501–510.
- [46] Alessandro Mantovani, Simone Aonzo, Yanick Fratantonio, and Davide Balzarotti. 2022. RE-Mind: a First Look Inside the Mind of a Reverse Engineer. In *31st USENIX Security Symposium (USENIX Security 22)*. 2727–2745.

²<https://github.com/E0HYL/DREAM-drift-adapt>

- [47] E Mariconti, L Onwuzurike, P Andriotis, E De Cristofaro, G Ross, and G Stringhini. 2017. MamaDroid: Detecting Android Malware by Building Markov Chains of Behavioral Models. In *Proceedings of the Network and Distributed Systems Security Symposium (NDSS)*.
- [48] Marc Masana, Idoia Ruiz, Joan Serrat, Joost van de Weijer, and Antonio M Lopez. 2018. Metric learning for novelty and anomaly detection. *arXiv preprint arXiv:1808.05492* (2018).
- [49] Shae McFadden, Zeliang Kan, Lorenzo Cavallaro, and Fabio Pierazzi. 2023. Poster: RPAL-Recovering Malware Classifiers from Data Poisoning using Active Learning. In *Proc. of ACM Conference on Computer and Communications Security (CCS)*.
- [50] Niall McLaughlin, Jesus Martinez del Rincon, BooJoong Kang, Suleiman Yerima, Paul Miller, Sakir Sezer, Yeganeh Safaei, Erik Tricket, Ziming Zhao, Adam Doupe, et al. 2017. Deep android malware detection. In *Proceedings of the seventh ACM on conference on data and application security and privacy*. 301–308.
- [51] Dongyu Meng and Hao Chen. 2017. Magnet: a two-pronged defense against adversarial examples. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*. 135–147.
- [52] Zhaoyi Meng, Jiale Zhang, Jiaqi Guo, Wansen Wang, Wenchao Huang, Jie Cui, Hong Zhong, and Yan Xiong. 2024. Detecting Android Malware by Visualizing App Behaviors from Multiple Complementary Views. *arXiv preprint arXiv:2410.06157* (2024).
- [53] Yin Minn Pa Pa, Shunsuke Tanizaki, Tetsui Kou, Michel Van Eeten, Katsunari Yoshioka, and Tsutomu Matsumoto. 2023. An attacker's dream? exploring the capabilities of chatgpt for developing. In *Proceedings of the 16th Cyber Security Experimentation and Test Workshop*. 10–18.
- [54] Tim Pearce, Alexandra Brintup, and Jun Zhu. 2021. Understanding softmax confidence and uncertainty. *arXiv preprint arXiv:2106.04972* (2021).
- [55] Feargus Pendlebury, Fabio Pierazzi, Roberto Jordaney, Johannes Kinder, and Lorenzo Cavallaro. 2019. TESSERACT: Eliminating experimental bias in malware classification across space and time. In *28th USENIX Security Symposium (USENIX Security 19)*. 729–746.
- [56] Lukas Pirch, Alexander Warnecke, Christian Wressnegger, and Konrad Rieck. 2021. Tagvet: Vetting malware tags using explainable machine learning. In *Proceedings of the 14th European Workshop on Systems Security*. 34–40.
- [57] Dima Rabadi and Sin G Teo. 2020. Advanced windows methods on malware detection and classification. In *Annual Computer Security Applications Conference*. 54–68.
- [58] Fereshteh Razmi and Li Xiong. 2023. Classification auto-encoder based detector against diverse data poisoning attacks. In *IFIP Annual Conference on Data and Applications Security and Privacy*. Springer, 263–281.
- [59] Pengcheng Ren, Chaoshun Zuo, Xiaofeng Liu, Wenrui Diao, Qingchuan Zhao, and Shanjing Guo. 2023. DEMISTIFY: Identifying On-device Machine Learning Models Stealing and Reuse Vulnerabilities in Mobile Apps. In *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 468–480.
- [60] Andrew Slavin Ross, Michael C Hughes, and Finale Doshi-Velez. 2017. Right for the right reasons: Training differentiable models by constraining their explanations. *arXiv preprint arXiv:1703.03717* (2017).
- [61] Wojciech Samek, Alexander Binder, Grégoire Montavon, Sebastian Lapuschkin, and Klaus-Robert Müller. 2016. Evaluating the visualization of what a deep neural network has learned. *IEEE transactions on neural networks and learning systems* 28, 11 (2016), 2660–2673.
- [62] Ramprasaath R Selvaraju, Stefan Lee, Yilin Shen, Hongxia Jin, Shalini Ghosh, Larry Heck, Dhruv Batra, and Devi Parikh. 2019. Taking a hint: Leveraging explanations to make vision and language models more grounded. In *Proceedings of the IEEE/CVF international conference on computer vision*. 2591–2600.
- [63] Michael R Smith, Nicholas T Johnson, Joe B Ingram, Armida J Carbajal, Bridget I Haus, Eva Domschot, Ramyaa Ramyaa, Christopher C Lamb, Stephen J Verzi, and W Philip Kegelmeyer. 2020. Mind the gap: On bridging the semantic gap between machine learning and malware analysis. In *Proceedings of the 13th ACM Workshop on Artificial Intelligence and Security*. 49–60.
- [64] Wolfgang Stammer, Patrick Schramowski, and Kristian Kersting. 2021. Right for the right concept: Revising neuro-symbolic concepts by interacting with their explanations. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 3619–3629.
- [65] Blake E Strom, Andy Applebaum, Doug P Miller, Kathryn C Nickels, Adam G Pennington, and Cody B Thomas. 2018. Mitre att&ck: Design and philosophy. In *Technical report*. The MITRE Corporation.
- [66] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. 2017. Axiomatic attribution for deep networks. In *International conference on machine learning*. PMLR, 3319–3328.
- [67] Stefano Teso and Kristian Kersting. 2019. Explanatory interactive machine learning. In *Proceedings of the 2019 AAAI/ACM Conference on AI, Ethics, and Society*. 239–245.
- [68] Bill Toulas. 2023. Ten new Android banking trojans targeted 985 bank apps in 2023. <https://www.bleepingcomputer.com/news/security/ten-new-android-banking-trojans-targeted-985-bank-apps-in-2023/>
- [69] Daniele Ucci, Leonardo Aniello, and Roberto Baldoni. 2019. Survey of machine learning techniques for analysis. *Computers & Security* 81 (2019), 123–147.
- [70] Liu Wang, Ren He, Haoyu Wang, Pengcheng Xia, Yuanchun Li, Lei Wu, Yajin Zhou, Xiapu Luo, Yulei Sui, Yao Guo, et al. 2021. Beyond the virus: a first look at coronavirus-themed Android malware. *Empirical Software Engineering* 26, 4 (2021), 82.
- [71] Liu Wang, Haoyu Wang, Ren He, Ran Tao, Guozhu Meng, Xiapu Luo, and Xuanzhe Liu. 2022. MalRadar: Demystifying Android malware in the new era. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 6, 2 (2022), 1–27.
- [72] Alexander Warnecke, Daniel Arp, Christian Wressnegger, and Konrad Rieck. 2020. Evaluating explanation methods for deep learning in security. In *2020 IEEE european symposium on security and privacy (EuroS&P)*. IEEE, 158–174.
- [73] Xian Wu, Wenbo Guo, Jia Yan, Baris Coskun, and Xinyu Xing. 2023. From Grim Reality to Practical Solution: Malware Classification in Real-World Noise. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 2602–2619.
- [74] Eular Xu. 2017. Xavier: An Information-Stealing Ad Library on Android. https://www.trendmicro.com/en_sg/research/17/t/analyzing-xavier-information-stealing-ad-library-android.html.
- [75] Ke Xu, Yingjiu Li, Robert Deng, Kai Chen, and Jiayun Xu. 2019. Droidevolver: Self-evolving android malware detection system. In *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 47–62.
- [76] Hongyu Yang, Youwei Wang, Liang Zhang, Xiang Cheng, and Ze Hu. 2024. A novel Android malware detection method with API semantics extraction. *Computers & Security* 137 (2024), 103651.
- [77] Jingkan Yang, Kaiyang Zhou, Yixuan Li, and Ziwei Liu. 2021. Generalized out-of-distribution detection: A survey. *arXiv preprint arXiv:2110.11334* (2021).
- [78] Limin Yang, Wenbo Guo, Qingying Hao, Arridhana Ciptadi, Ali Ahmadzadeh, Xinyu Xing, and Gang Wang. 2021. CADE: Detecting and Explaining Concept Drift Samples for Security Applications. In *30th USENIX Security Symposium (USENIX Security 21)*. 2327–2344.
- [79] Jiahao Yu, Wenbo Guo, Qi Qin, Gang Wang, Ting Wang, and Xinyu Xing. 2023. {AIRS}: Explanation for Deep Reinforcement Learning based Security Applications. In *32nd USENIX Security Symposium (USENIX Security 23)*. 7375–7392.
- [80] Xiaohan Zhang, Yuan Zhang, Ming Zhong, Daizong Ding, Yinzi Cao, Yukun Zhang, Mi Zhang, and Min Yang. 2020. Enhancing state-of-the-art classifiers with API semantics to detect evolved android malware. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 757–770.

A Malware Dataset

As shown in Table 2, we use two datasets for malware family classification. Specifically, we select 8 families for each of the Drebin [78] and the MalRadar [71] dataset, where the family selection adheres to the same criteria used in the CADE paper. The Drebin dataset spans the years from 2010 to 2012, offering insights into malware characteristics from the early period. To capture more recent trends in the evolving malware landscape, the Malradar dataset [71] covers the period from 2015 to 2021. We use the malicious behaviors defined in the Malradar paper. To ensure consistency in behavioral analysis, we augment the Drebin dataset through extrapolating the labels based on Malradar's definitions, supplemented by expert analysis and validation through GPT [53]. Note that the concept labels were assigned with minimal additional effort, and they are uniform within each family. In cases where certain behaviors in the Malradar dataset are specific to particular variants, we accommodate this by setting the relevant valid label mask m_e to zero. Despite the simplicity, this approach is in line with an active learning setting that typically utilizes limited human effort.

B Lessons from Current Detectors

Despite the introduction of methods for drift detection in existing research, a comprehensive comparison focusing exclusively on their drift detection performance remains absent. This shortfall leads to a gap in understanding the true effectiveness of these methods under different conditions. Notably, the most recent work HCC has been recognized for its effectiveness in updating malware detection classifiers. This method is exclusively applicable to intra-class drifts. To address the gap, our study begins by examining the problem within an intra-class scenario. In the following, we establish a methodology for evaluating intra-class drift detection performance and provide several insights.

Evaluation Method. For malware detection tasks, we leverage malware samples from the Malradar dataset [71], which covers 4,410 malware across 180 families including singletons. Moreover, we make efforts to collect 43,641 benign samples in the same year period from 2015 to 2020. Our dataset has been carefully designed to mitigate spatial bias and temporal bias [55]. For the classifier, we use the DREBIN features and an MLP model (detailed in Section 5.1). Regarding baselines, we take into account the vanilla probability-based detector and the three innovative detectors in Table 1. During the evaluation phase, we assign positive drift labels to samples where predictions are incorrect, following the rationale that these misclassified samples are particularly valuable for adapting the classifier. This approach differs from inter-class drift detection evaluation, where positive drift labels are typically assigned based on the presence of unseen families, as utilized in existing work [78].

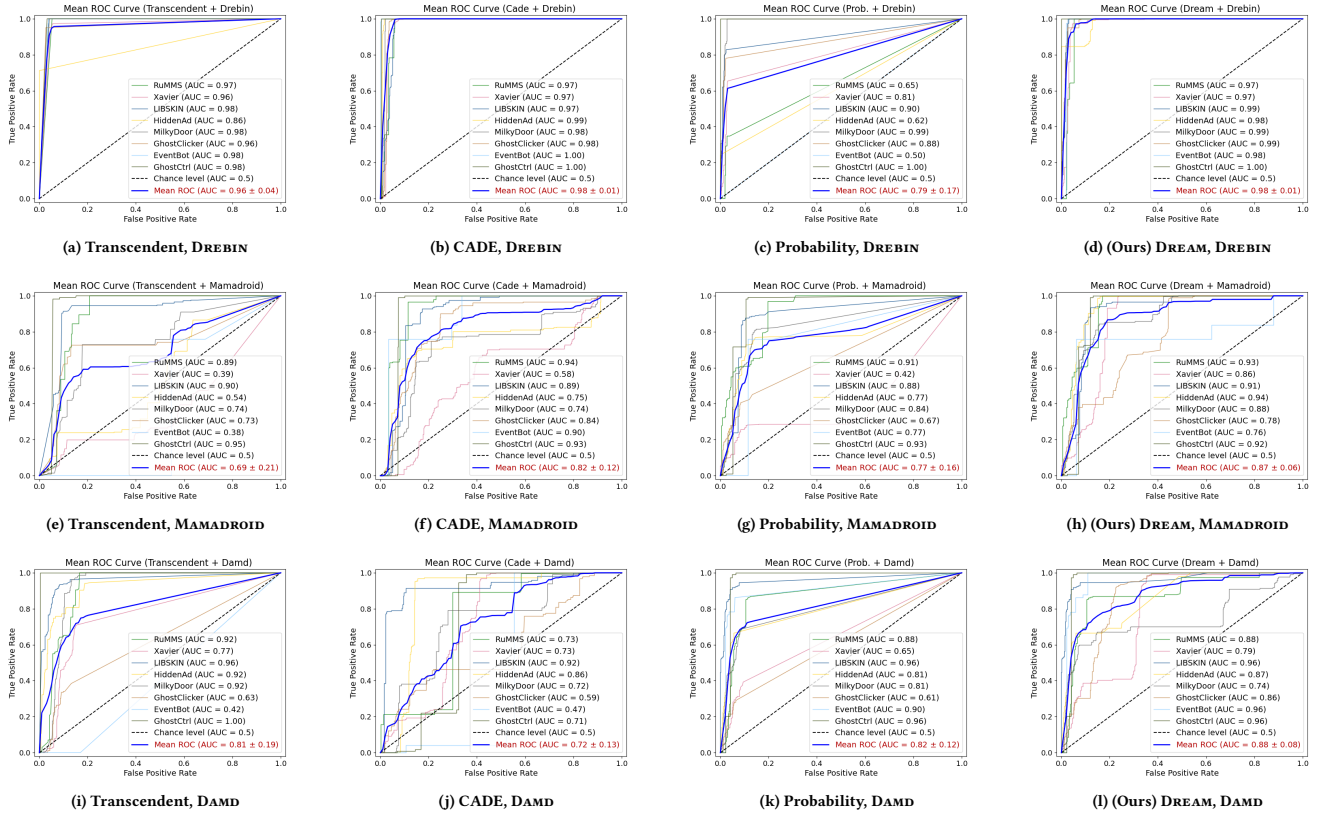


Figure 7: Evaluation of inter-class drift detection on Malradar dataset with three feature spaces. The first three columns are Transcendent, CADE, Probability, respectively, and our method is on the last column.

	2016	2017	2018	2019	2020	Avg.
Prob.	0.631	0.646	0.600	0.696	0.662	0.647
Trans.	0.733	0.674	0.587	0.708	0.599	0.660
CADE	0.721	0.686	0.739	0.594	0.487	0.645
HCC	0.708	0.649	0.605	0.719	0.654	0.667
HCC _{ce}	0.711	0.649	0.606	0.719	0.655	0.668
HCC _{hc}	0.581	0.609	0.592	0.678	0.657	0.623

Table 7: Intra-class drift detection performance of existing works. For the state-of-the-art method HCC, we also make a separate study of the two items in its drift scoring function.

Explainer	CBP	DRR
Random	0.017 ± 0.031	0.023 ± 0.009
Dri-IG	0.228 ± 0.103	0.355 ± 0.337
CADE	0.173 ± 0.112	0.974 ± 0.009
DREAM	0.331 ± 0.197	0.974 ± 0.009

Table 8: Evaluation of feature-level explanations.

Then we compute the AUC metric using drifting scores and the ground truth drift labels.

Key Findings. Our evaluation results are shown in Table 7, from which we have identified four key findings. 1) The performance order in our experiments, i.e., HCC, Transcendent, and Probability

all outperforming CADE, highlights the essential role of *model sensitivity* in effective drift detection. 2) Although CADE is not specifically tailored for intra-class drift, its superior performance in a non-model sensitive comparison with the HCC_{hc} indicates the potential advantages of *data autonomy* and its *autoencoder-based structure*. 3) HCC surpassing Transcendent in performance indicates that *learning a distance metric*, rather than relying on calibration, can be more effective in complex models like DNNs. 4) The performance decline over time, particularly noticeable in the CADE method, suggests that *continuous adaptation* might be necessary not just for classifiers, but also for detection models.

C Drift Explanation Metric

In the drift explanation context, CBP is quantified as the proportion of training set samples with higher drifting scores than the perturbed samples, formally represented as:

$$\frac{|\{\alpha \in \mathcal{D}_{\text{train}}[\hat{y}_i] : u_d(\alpha; \mathbf{M}) \geq u_d(\mathbf{x}'_d; \mathbf{M})\}|}{|\mathcal{D}_{\text{train}}[\hat{y}_i]|} \quad (9)$$

DRR is defined as the ratio of the reduced distance to the original distance between the drifting sample and the reference sample. It is worth mentioning that CBP is our primary metric of interest, as it directly correlates with classifier results, whereas DRR serves more as a supplementary measure in the detector's latent space.